

# Отладка производительности

Одной из значительных проблем производительности приложения является медленное построение страниц и открытие диалоговых окон. Чтобы выяснить причину плохой отзывчивости интерфейса на действия пользователя, необходимо профилировать.



По умолчанию у методов ps сервисов стоит отсечка **3 секунды**. Если метод выполнялся без ошибок менее, чем за 3 секунды, то график загрузки не построится. Чтобы посмотреть график загрузки на рабочем сайте, попросите настроить отсечку в 1 секунду или меньше у метода, который вас интересует.

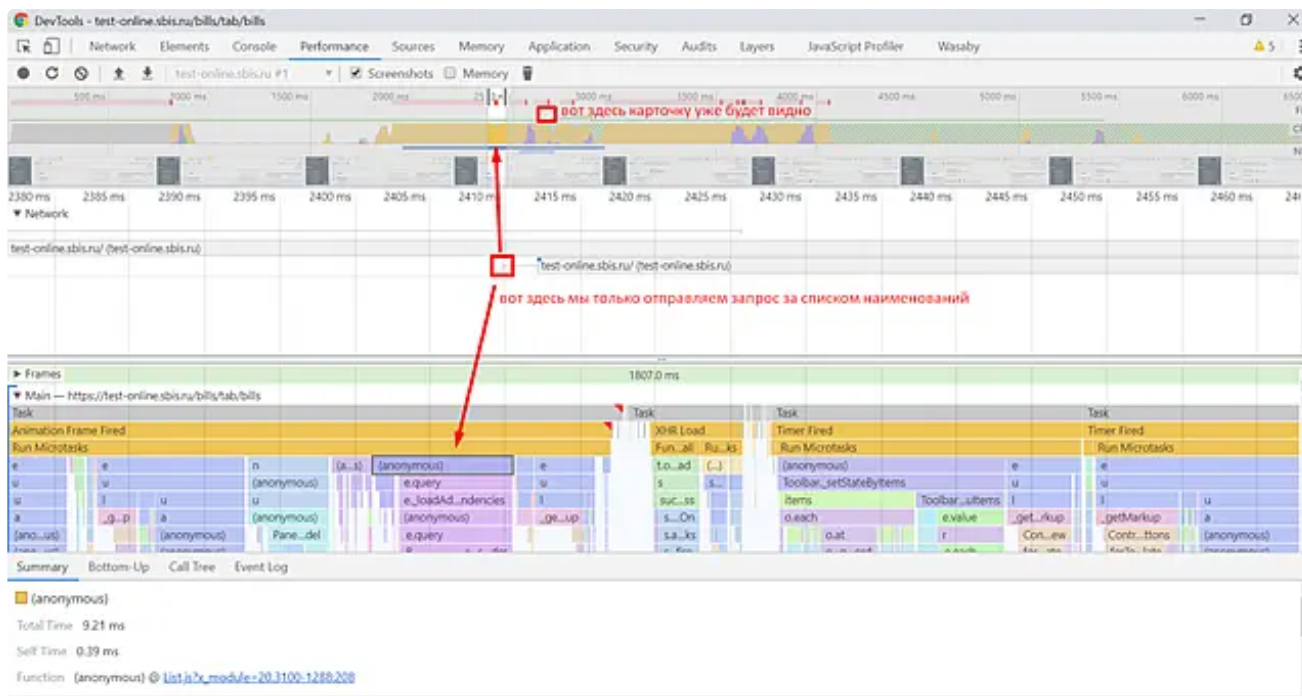
Также для правки настройки на рабочем сайте можно передать в запрос `true` в заголовок `X-LogEntireTask`.

Далее рассмотрим основные причины плохой производительности и способы их устранения.

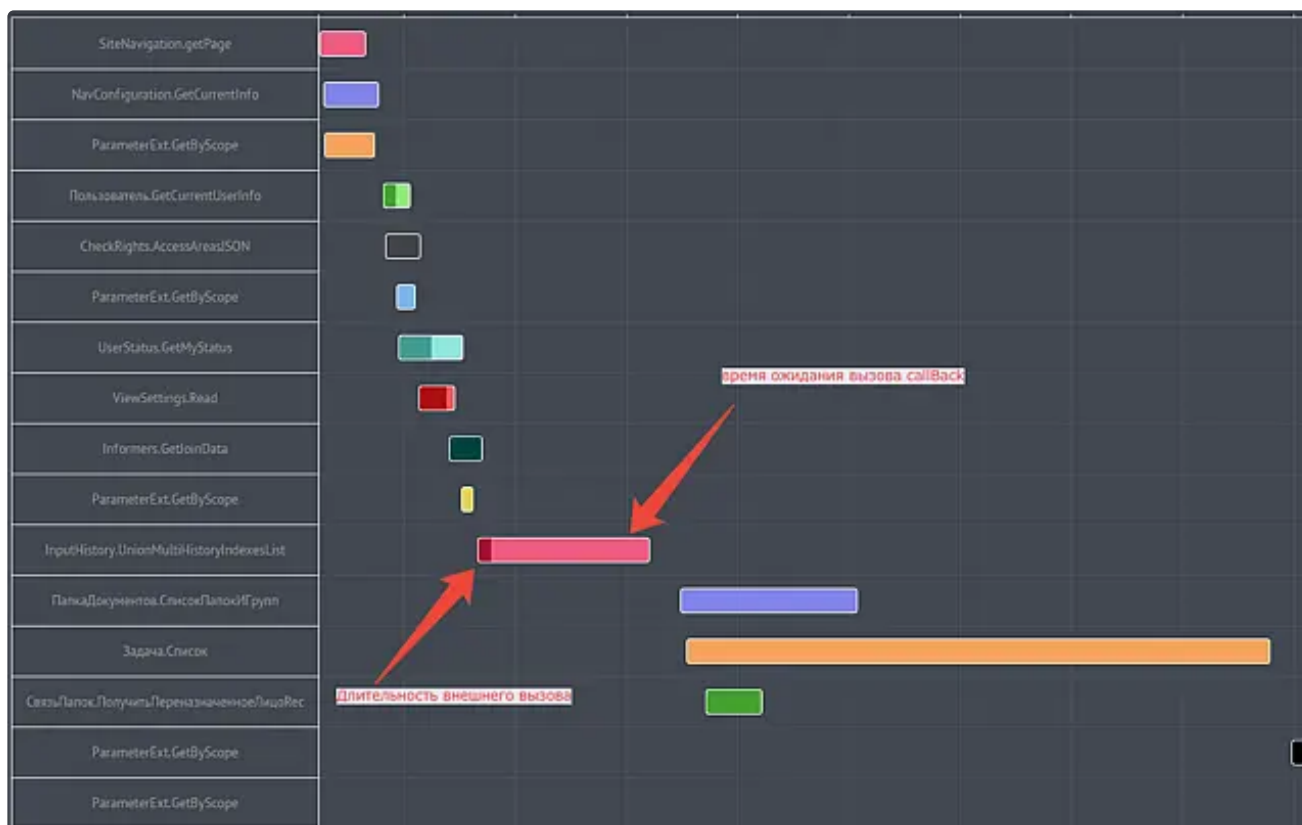
## Последовательные асинхронные запросы

Зачастую возникает необходимость загрузить данные для нескольких независимых контролов. Каждый контрол при этом загружает свои данные и ничего не знает о соседях. Это приводит к тому, что запросы на БЛ откладываются на время построения родительских контролов, а общая длительность построения будет высокой.

На клиенте данную проблему можно диагностировать с помощью инструмента [Chrome DevTools](#). Например:



Диагностировать данную проблему на [Сервисе представления](#) следует при помощи [графика вызовов методов БЛ](#):



**Решение проблемы** заключается в переходе к **модели параллельных запросов**, которые запускаются сразу после действия пользователя. Таким образом, запрос за данными выполнится в самом начале построения, и в прикладные контролы будут переданы уже загруженные данные.

При построении страниц применяйте технологию [pagex](#).

При построении диалоговых окон используйте опенер [SabyPage/pageOpener:Opener](#). Вызовите метод [open](#) и передайте в него **pageId** - начальную страницу, которую нужно открыть.

```

1 import('SabyPage/pageOpener').then((pageOpener) => {
2   const opener = new pageOpener.Opener();
3   opener.open({
4     opener: parentControl,
5     pageId: 'myPageId',
6     scopeId: 'app-scope-id',
7     queryParams: {foo: 'bar'}
8   });
9 });

```

🔍 Подробнее о получении логов, в том числе с **боевого** облака, см. в статье [Построение графика БЛ методов](#).

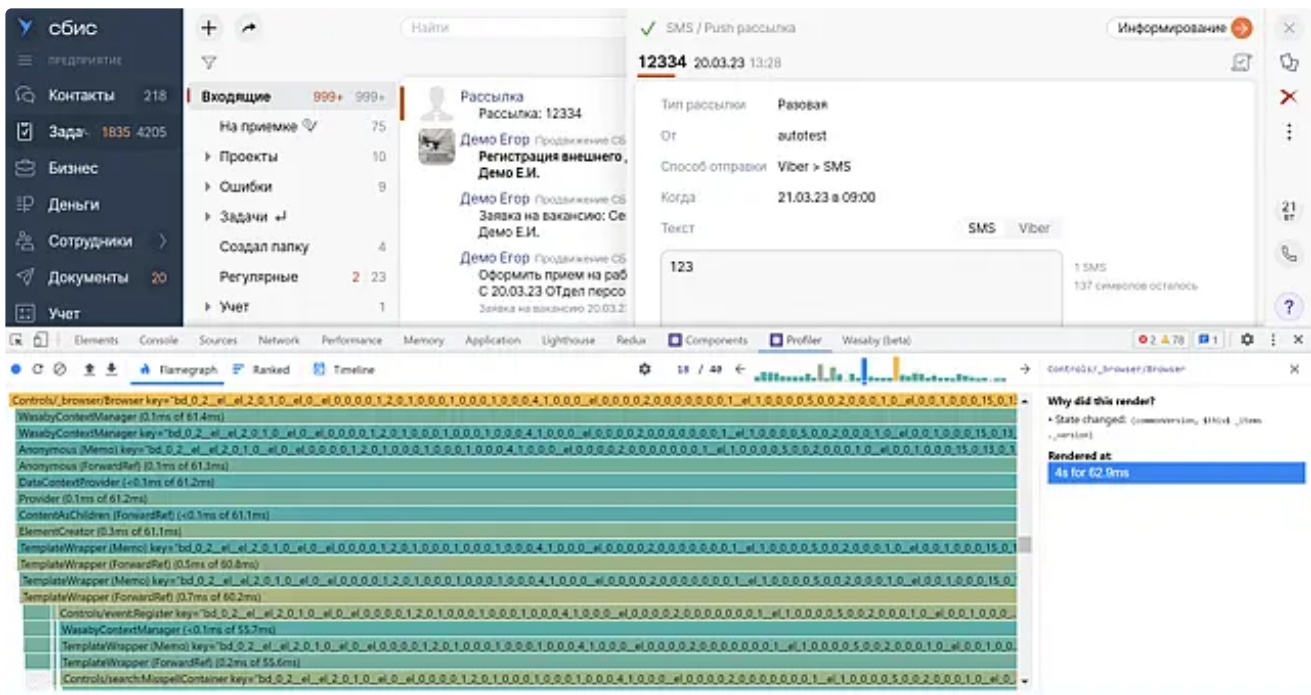
## Лишние синхронизации

Лишние синхронизации зачастую являются следствием [описанной выше проблемы](#) и связаны с запутанным потоком распространения данных. В результате момент "оживления" страницы откладывается до последней синхронизации.

Еще одной причиной лишних синхронизаций может служить проблема расчета размеров элементов в готовой верстке, когда за первый цикл синхронизации элементы выводятся в DOM, затем считываются их размеры и с учетом полученных размеров происходит пересинхронизация.

Подробнее о причинах появления лишних синхронизаций читайте в статье [Поиск причин перерисовки](#).

Синхронизации можно отслеживать с помощью инструмента [React Developer Tools](#).



Чтобы избежать лишних синхронизаций при построении страницы, необходимо:

- Организовать **однонаправленный поток распространения данных** по принципу "сверху-вниз" от родительского компонента к дочерним. Так все данные будут получены до начала рендеринга, а затем с помощью опций будут доставлены до нужного компонента.
- Не изменять состояние в эффектах (**useEffect** или **useLayoutEffect**) сразу после построения. Во многих ситуациях эффект можно заменить другими вещами (например, **useMemo**). В официальной документации есть [подробная статья на эту тему](#).
- Не изменять состояние компонента в **\_afterMount**. Корректное состояние компонента должно устанавливаться в **\_beforeMount**.



Избавляться необходимо от ВСЕХ лишних синхронизаций, даже если **React Developer Tools** показывает продолжительность в несколько мс. Реальное время работы платформы и браузера может оказаться больше.

## Медленная реакция на действия пользователя

При взаимодействии пользователя с контентом очень важна обратная связь - время реакции сайта на действия пользователя не должно быть большим. Однако в некоторых случаях даже наличие одного единственного запроса на БЛ является проблемой.

Например, создание счета в приложении [Мой Склад](#) занимало примерно столько же времени, сколько занимало выполнение запроса на БЛ. В результате было решено, что карточку создания счета нужно показывать сразу после клика по кнопке "+", а вызов метода БЛ производить параллельно рендерингу карточки.

Чтобы начать построение карточки, не дожидаясь ответа БЛ, используйте [механизм двухфазного открытия окна](#). Такой способ построения настраивается при помощи опции [initializingWay](#) со значениями:

- **delayedRead** - при открытии диалога для редактирования записи.
- **delayedCreate** - при открытии диалога для создания новой записи.

В обоих случаях верстка контрола строится по записи, переданной в опцию [record](#). Параллельно с этим запрашиваются данные с БЛ, после получения которых диалог обновляется.

Этот способ построения является компромиссом между скоростью открытия диалога и временем подготовки отображаемых данных.

Пример:

```
1 import { INITIALIZING_WAY, Controller } from 'Controls/form';
2
3 function Component() {
4   return <Controller initializingWay=
5     {INITIALIZING_WAY.DELAYED_CREATE} />
6 }
```

Еще один способ ускорить отзывчивость интерфейса на действия пользователя - выполнять открытие диалога по событию **mouseDown**, что позволит открывать диалог, когда кнопка мыши зажата над элементом.



Этот способ можно использовать только при отсутствии [Drag-n-Drop](#).

## Оптимизация загрузки статических ресурсов

Чтобы отрисовка страницы происходила быстро, необходимо **оптимизировать загрузку статических ресурсов при старте приложения** - не загружать исходники компонентов, которые не видны на странице.

Однако, существует проблема - чем меньше статических ресурсов будет загружено при открытии реестра, тем больше ресурсов потребуются загрузить при первом открытии диалогового окна.

Для решения проблемы необходимо [проанализировать исходные файлы на неиспользуемый код](#) и убедиться, что не загружаются исходные данные компонентов, которые не видны на странице.

Также необходимо настроить [предзагрузку статических ресурсов](#), что улучшит отзывчивость интерфейса на действия пользователя и ускорит открытие диалога.

## Виртуальный скролл в списках

Если в вашем интерфейсе есть списки с большим количеством отображаемых элементов, ознакомьтесь с [руководством по оптимизации списка с помощью виртуального скролла](#). Опции, описанные в статье, необходимо настраивать для того, чтобы списки не запрашивали и не отрисовывали лишние элементы.

Чтобы правильно подобрать параметры опций, используйте данные о [размере рабочей области](#).